

Proyecto 3. Reconocimiento de mensajes spam.

José R. Rodríguez

Departamento de Ingeniería en Ciencias de la Computación y Tecnología,
Facultad de Ingeniería, Universidad del Valle de Guatemala; Instructor: Javier
Fong

Resultados

Cuadro 1: Matriz de confusión del modelo.

Real Predicción	Ham	Spam
Ham	957	7
Spam	16	133

Cuadro 2: Métricas del modelo.

Etiqueta	Precision	Recall	F1 score
Ham	0.98	0.99	0.99
Spam	0.95	0.89	0.92

Discusión de resultados.

En el cuadro 1 y 2, se observa que el modelo es mucho mejor para etiquetar aquellos correos que son ham a aquellos que son spam, esto se priorizó debido a que para los usuarios es preferible que aquellos correos que ellos consideran importantes no sean etiquetados como spam, aunque eso suponga que algunos correos spam no se reconozcan.

En este modelo fue necesario realizar tokenización, para separar las palabras y signos de puntuación con el fin de operar individualmente cada componente de las frases. Se convirtió todo a minúscula para evitar que palabras escritas entre mayúsculas y minúsculas a pesar de ser las mismas, sean tomadas como distintas debido al código ASCII. Se removieron signos de puntuación, debido al poco aporte que pueden dar en un modelo de Bayes Ingenuo, ya que nos interesa la frecuencia y no el orden. Se quitaron las stop-words, ya que son palabras muy comunes que no suelen dar ningún significado a los temas de los cuales se hablan, ya que suelen ser conectores o determinantes. Por último, en el proceso de limpieza, se realizó una lematización, debido a que se desea evitar que palabras con significados similares, como verbos conjugados, sean tomadas como distintas, ya que este proceso toma el significado gramatical de cada palabra.

Para construir el modelo, se utilizó TfidfVectorizer con el objetivo de transformar los textos en representaciones numéricas basadas en la importancia de las palabras dentro del conjunto de documentos. Además, se usó `token_pattern=r"(?u)\b\w+\b"` para definir qué elementos del texto serían considerados tokens, permitiendo capturar palabras, números y términos de una sola letra. También se estableció `ngram_range=(1, 2)`, lo que considera palabras individuales y combinaciones de dos palabras consecutivas, capturando algunas relaciones entre palabras debido al orden y palabras compuestas. Se utilizó `min_df=1`, para conservar palabras que aparecieran al menos en un documento, y `max_df=0.8`, la cual elimina palabras que aparecen en más del 80% de los documentos. Asimismo, se activó `sublinear_tf=True` para reducir el efecto de palabras repetidas muchas veces dentro de un mismo texto, evitando que dominaran excesivamente la representación. Finalmente, se mantuvo `use_idf=True` y `smooth_idf=True` para aplicar el peso inverso de frecuencia documental de forma estable, y `norm="l2"` para normalizar los vectores, facilitando la comparación entre documentos de distinta longitud.

Repositorio

https://github.com/ricrod01/PROYECTO_3_IA.git

Referencias

Manning, C. D., Raghavan, P., & Schütze, H. (2008). Introduction to information retrieval. Cambridge University Press.

Pedregosa, F. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.